

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 6”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat-Nya, sehingga laporan praktikum Praktikum Struktur Data untuk pertemuan pekan ke-6 ini dapat diselesaikan. Fokus utama pada pertemuan kali ini adalah pemahaman dan implementasi Double Linked List (DLL).

Laporan ini mendokumentasikan proses pembuatan simpul dengan dua penunjuk (pointer), teknik penelusuran dua arah, serta manipulasi data melalui operasi penambahan dan penghapusan simpul. Penulis berharap laporan ini dapat menjadi referensi yang bermanfaat dalam memahami struktur data dinamis yang lebih kompleks.

Padang, 14 Mei, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Class NodeDLL_2511532014.....	3
2.2 Class insertDLL_2511532014.....	3
2.3 Class HapusDLL_2511532014.....	6
2.4 Class PenelusuranDLL_2511532014.....	9
BAB III KESIMPULAN	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Double Linked List (DLL) adalah pengembangan dari Single Linked List di mana setiap simpul (node) memiliki dua buah referensi atau penunjuk (pointer). Satu penunjuk merujuk ke simpul berikutnya (next), dan satu penunjuk lainnya merujuk ke simpul sebelumnya (prev). Struktur ini memungkinkan pergerakan data dilakukan secara dua arah, baik maju maupun mundur.

Meskipun membutuhkan alokasi memori yang sedikit lebih besar untuk menyimpan penunjuk tambahan, DLL menawarkan fleksibilitas yang lebih tinggi dibandingkan SLL, terutama saat melakukan operasi penghapusan simpul atau penelusuran balik tanpa harus mengulang dari simpul awal. Praktikum pekan ke-6 ini bertujuan untuk mempraktikkan secara langsung cara kerja mekanisme penunjuk ganda tersebut dalam bahasa pemrograman Java.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami struktur dasar simpul pada Double Linked List yang memiliki penunjuk next dan prev.
2. Mampu mengimplementasikan penelusuran data secara maju (forward) dan mundur (backward).
3. Mampu melakukan operasi penyisipan data di awal, akhir, dan posisi tertentu pada DLL.
4. Mampu melakukan operasi penghapusan simpul pada posisi head, tail, maupun posisi tengah.
5. Melatih logika pengelolaan referensi objek agar tidak terjadi kehilangan data saat pemutusan rantai simpul dilakukan.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Fleksibilitas Navigasi: Mahasiswa memahami cara melakukan iterasi data dari dua arah yang berbeda secara efisien.

2. Optimasi Operasi: Memahami bahwa penghapusan simpul tertentu pada DLL bisa lebih cepat karena referensi simpul sebelumnya sudah tersedia di dalam simpul itu sendiri.
3. Penguasaan Struktur Data Dinamis: Meningkatkan kemampuan dalam mengelola struktur data yang ukurannya dapat berubah secara fleksibel di memori

BAB II PEMBAHASAN

2.1 Class NodeDLL_2511532014

2.1.1 Langkah Pengerjaan

1. Mendefinisikan variabel data_2014 untuk menyimpan nilai.
2. Mendeklarasikan dua objek referensi: next_2014 (ke depan) dan prev_2014 (ke belakang).
3. Membuat konstruktor untuk menginisialisasi data dan mengatur kedua penunjuk ke null sebagai standar simpul baru.

2.1.2 Kode Program

```
1 package pekan6_2511532014;
2
3 public class NodeDLL_2511532014 {
4     // mendefinisikan kelas node
5     int data_2014;
6     NodeDLL_2511532014 next_2014; //pointer ke next node
7     NodeDLL_2511532014 prev_2014; //pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511532014(int data_2014) {
11        this.data_2014 = data_2014;
12        this.next_2014 = null;
13        this.prev_2014 = null;
14    }
15 }
16
```

2.1.3 Analisis

Kelas ini adalah komponen fundamental DLL. Berbeda dengan SLL, penambahan variabel prev_2014 memungkinkan setiap simpul "mengetahui" siapa tetangganya di kedua sisi. Hal ini adalah kunci utama yang memungkinkan operasi navigasi dua arah dijalankan.

2.2 Class insertDLL_2511532014

2.2.1 Langkah Pengerjaan

1. Metode insertBegin_2014: Membuat simpul baru, menghubungkan next-nya ke head lama, dan mengatur prev pada head lama ke simpul baru.

2. Metode insertEnd_2014: Mencari simpul terakhir, lalu menyambungkan penunjuk secara timbal balik antara simpul terakhir lama dan simpul baru.
3. Metode insertAfter_2014: Menyisipkan simpul di tengah dengan memperbarui empat penunjuk sekaligus (dua pada simpul baru, satu pada simpul sebelumnya, dan satu pada simpul sesudahnya).

2.2.2 Kode Program

```

1 package pekan6_2511532014;
2
3 public class insertDLL_2511532014 {
4     //menambahkan Node di awal dll
5     static NodeDLL_2511532014 insertBegin_2014(NodeDLL_2511532014
6     head_2014, int data_2014) {
7         //buat node baru
8         NodeDLL_2511532014 new_node_2014 = new
9         NodeDLL_2511532014(data_2014);
10        //jadikan pointer nextnya head
11        new_node_2014.next_2014 = head_2014;
12        //jadikan pointer prev head ke new
13        if (head_2014 != null) {
14            head_2014.prev_2014 = new_node_2014;
15        }
16        return new_node_2014;
17    }
18    // fungsi menambahkan node di akhir
19    public static NodeDLL_2511532014
20    insertEnd_2014(NodeDLL_2511532014 head_2014, int newData_2014) {
21        //buat node baru
22        NodeDLL_2511532014 newNode_2014 = new
23        NodeDLL_2511532014(newData_2014);
24        //jika dll null jadikan head
25        if (head_2014 == null) {
26            head_2014 = newNode_2014;
27        } else {
28            NodeDLL_2511532014 curr_2014 = head_2014;
29            while (curr_2014.next_2014 != null) {
30                curr_2014 = curr_2014.next_2014;
31            }
32            curr_2014.next_2014 = newNode_2014;
33            newNode_2014.prev_2014 = curr_2014;
34        }
35        return head_2014;
36    }
37    //fungsi menambahkan node di posisi tertentu
38    public static NodeDLL_2511532014
39    insertAtPosition_2014(NodeDLL_2511532014 head_2014, int pos_2014,
40    int new_data_2014) {
41        //buat node baru
42        NodeDLL_2511532014 new_node_2014 = new NodeDLL_2511532014
43        (new_data_2014);
44        if (pos_2014 == 1) {
45            new_node_2014.next_2014 = head_2014;
46            if (head_2014 != null) {
47                head_2014.prev_2014 = new_node_2014;

```

```

41         }
42         head_2014 = new_node_2014;
43         return head_2014;
44     }
45     NodeDLL_2511532014 curr_2014 = head_2014;
46     for (int i = 1; i < pos_2014 - 1 && curr_2014 != null;
47 ++i ) {
48         curr_2014 = curr_2014.next_2014;
49     }
50     if (curr_2014 == null) {
51         System.out.println("Posisi tidak ada");
52         return head_2014;
53     }
54     new_node_2014.prev_2014 = curr_2014;
55     new_node_2014.next_2014 = curr_2014.next_2014;
56     curr_2014.next_2014 = new_node_2014;
57     if (new_node_2014.next_2014 != null) {
58         new_node_2014.next_2014.prev_2014 = new_node_2014;
59     }
60     return head_2014;
61 }
62 public static void printList_2014(NodeDLL_2511532014 head_2014)
63 {
64     NodeDLL_2511532014 curr_2014 = head_2014;
65     while (curr_2014 != null) {
66         System.out.print(curr_2014.data_2014 + " <-> ");
67         curr_2014 = curr_2014.next_2014;
68     }
69     System.out.println();
70 }
71 public static void main(String[] args) {
72     //membuat dll 2 <-> 3 <-> 5
73     NodeDLL_2511532014 head_2014 = new NodeDLL_2511532014(2);
74     head_2014.next_2014 = new NodeDLL_2511532014(3);
75     head_2014.next_2014.prev_2014 = head_2014;
76     head_2014.next_2014.next_2014 = new
77     NodeDLL_2511532014(5);
78     head_2014.next_2014.next_2014.prev_2014 =
79     head_2014.next_2014;
80     //cetak DLL awal
81     System.out.print("DLL Awal: ");
82     printList_2014(head_2014);
83     // tambah 1 diawal
84     head_2014 = insertBegin_2014(head_2014, 1);
85     System.out.print("simpul 1 ditambah di awal: ");
86     printList_2014(head_2014);
87     // tambah 6 di akhir
88     System.out.print("simpul 6 ditambah di akhir: ");
89     int data_2014 = 6;
90     head_2014 = insertEnd_2014(head_2014, data_2014);
91     printList_2014(head_2014);
92     // mena,bah node 4 diposisi 4
93     System.out.print("tambah node 4 diposisi 4: ");
94     int data2_2014 = 4;
95     int pos_2014 = 4;

```



```

93         head_2014 = insertAtPosition_2014(head_2014, pos_2014,
data2_2014);
94         printList_2014(head_2014);
95     }
96
97 }
98

```

2.2.3 Output

```

<terminated> insertDLL_2511532014 [Java Application] C:\Users\asus\p2\pool\plugins\org.eclipse.
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 diposisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

2.2.4 Analisis

Operasi penyisipan pada DLL menuntut ketelitian tinggi. Jika salah satu penunjuk (misalnya prev pada simpul sesudahnya) tidak diperbarui, maka rantai DLL akan rusak saat dilakukan penelusuran mundur.

2.3 Class HapusDLL_2511532014

2.3.1 Langkah Pengerjaan

1. delHead_2014: Memindahkan head ke simpul selanjutnya dan memutus hubungan prev ke null.
2. delLast_2014: Menelusuri hingga simpul terakhir, lalu memutuskan hubungan next pada simpul sebelumnya.
3. deleteNode_2014: Menghapus simpul di posisi tertentu dengan menghubungkan langsung simpul sebelum dan simpul sesudah dari simpul yang akan dihapus.

2.3.2 Kode Program

```

1  package pekan6_2511532014;
2
3  public class HapusDLL_2511532014 {
4      //fungsi menghapus node awal
5      public static NodeDLL_2511532014
delHead_2014(NodeDLL_2511532014 head_2014) {
6          if (head_2014 == null) {
7              return null;
8          }
9          head_2014 = head_2014.next_2014;
10         if (head_2014 != null) {
11             head_2014.prev_2014 = null;
12         }
13         return head_2014;
14     }
15     //fungsi menghapus diakhir

```

```

16     public static NodeDLL_2511532014
dellLast_2014(NodeDLL_2511532014 head_2014) {
17         if (head_2014 == null) {
18             return null;
19         }
20         if (head_2014.next_2014 == null) {
21             return null;
22         }
23         NodeDLL_2511532014 curr_2014 = head_2014;
24         while (curr_2014.next_2014 != null) {
25             curr_2014 = curr_2014.next_2014;
26         }
27         //update pointer previous node
28         if (curr_2014.prev_2014 != null) {
29             curr_2014.prev_2014.next_2014 = null;
30         }
31         return head_2014;
32     }
33     //fungsi menghapus node pada posisi tertentu
34     public static NodeDLL_2511532014 delPos_2014(NodeDLL_2511532014
head_2014, int pos_2014) {
35         //jika dll kosong
36         if (head_2014 == null) {
37             return head_2014;
38         }
39         NodeDLL_2511532014 curr_2014 = head_2014;
40         // telusuri sampai ke noe yang akan dihapus
41         for (int i = 1; curr_2014 != null && i < pos_2014; ++i) {
42             curr_2014 = curr_2014.next_2014;
43         }
44         //jika posisi tidak ditemukan
45         if (curr_2014 == null) {
46             return head_2014;
47         }
48         //update pointer
49         if (curr_2014.prev_2014 != null) {
50             curr_2014.prev_2014.next_2014 =
curr_2014.next_2014;
51         }
52         if (curr_2014.next_2014 != null) {
53             curr_2014.next_2014.prev_2014 =
curr_2014.prev_2014;
54         }
55         //jika yang dihapus head
56         if (head_2014 == curr_2014) {
57             head_2014 = curr_2014.next_2014;
58         }
59         return head_2014;
60     }
61     //fungsi mencetak DLL
62     public static void printList_2014(NodeDLL_2511532014 head_2014)
{
63         NodeDLL_2511532014 curr_2014 = head_2014;
64         while (curr_2014 != null) {
65             System.out.print(curr_2014.data_2014 + " ");
66             curr_2014 = curr_2014.next_2014;
67         }

```

```

68         System.out.println();
69     }
70
71
72     public static void main(String[] args) {
73         //buat sebuah dll
74         NodeDLL_2511532014 head_2014 = new NodeDLL_2511532014(1);
75         head_2014.next_2014 = new NodeDLL_2511532014(2);
76         head_2014.next_2014.prev_2014 = head_2014;
77         head_2014.next_2014.next_2014 = new
NodeDLL_2511532014(3);
78         head_2014.next_2014.next_2014.prev_2014 =
head_2014.next_2014;
79         head_2014.next_2014.next_2014.next_2014 = new
NodeDLL_2511532014(4);
80         head_2014.next_2014.next_2014.next_2014.prev_2014 =
head_2014.next_2014.next_2014;
81         head_2014.next_2014.next_2014.next_2014.next_2014 = new
NodeDLL_2511532014(5);
82
83         head_2014.next_2014.next_2014.next_2014.next_2014.prev_2014 =
head_2014.next_2014.next_2014.next_2014;
84
85         System.out.print("DLL awal: ");
86         printList_2014(head_2014);
87
88         System.out.print("Setelah head dihapus: ");
89         head_2014 = delHead_2014(head_2014);
90         printList_2014(head_2014);
91
92         System.out.print("Setelah node terakhir dihapus: ");
93         head_2014 = delLast_2014(head_2014);
94         printList_2014(head_2014);
95
96         System.out.print("Menghapus node ke 2: ");
97         head_2014 = delPos_2014(head_2014, 2);
98         printList_2014(head_2014);
99     }
100
101 }
102

```

2.3.3 Output

```

<terminated> HapusDLL_2511532014 [Java Application] C:\Users\asus\p2\pool\p
DLL awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
Menghapus node ke 2: 2 4

```

2.3.4 Analisis

Penghapusan simpul tengah pada DLL sangat efisien karena kita dapat mengakses simpul sebelumnya (prev) dan simpul sesudahnya (next) secara langsung dari simpul target. Hal ini meminimalkan kebutuhan penelusuran ulang untuk mencari simpul pendahulu.

2.4 Class PenelusuranDLL_2511532014

2.4.1 Langkah Pengerjaan

1. Implementasi forwardTraversal_2014 dengan bergerak dari head menggunakan penunjuk next.
2. Implementasi backwardTraversal_2014 dengan bergerak dari tail menggunakan penunjuk prev.

2.4.2 Kode Program

```
1 package pekan6_2511532014;
2
3 public class PenelusuranDLL_2511532014 {
4     //fungsi penelusuran maju
5     static void forwardTraversal_2014(NodeDLL_2511532014 head_2014)
6     {
7         //memulai penelusuran dari head
8         NodeDLL_2511532014 curr_2014 = head_2014;
9         //lanjutkan sampai akhir
10        while (curr_2014 != null) {
11            //print data
12            System.out.print(curr_2014.data_2014 + " <-> ");
13            //pindahkan ke node berikutnya
14            curr_2014 = curr_2014.next_2014;
15        }
16        //print spasi
17        System.out.println();
18    }
19    //fungsi penelusuran mundur
20    static void backwardTraversal_2014(NodeDLL_2511532014
21    tail_2014) {
22        //mulai dari akhir
23        NodeDLL_2511532014 curr_2014 = tail_2014;
24        //lanjut sampai head
25        while (curr_2014 != null) {
26            //cetak data
27            System.out.print(curr_2014.data_2014 + " <-> ");
28            // pindahkan ke node sebelumnya
29            curr_2014 = curr_2014.prev_2014;
30        }
31        //cetak spasi
32        System.out.println();
33    }
34    public static void main(String[] args) {
35        //cetak DLL
36        NodeDLL_2511532014 head_2014 = new NodeDLL_2511532014(1);
```

```

37         NodeDLL_2511532014 second_2014 = new
NodeDLL_2511532014(2);
38         NodeDLL_2511532014 third_2014 = new
NodeDLL_2511532014(3);
39         head_2014.next_2014 = second_2014;
40         second_2014.prev_2014 = head_2014;
41         second_2014.next_2014 = third_2014;
42         third_2014.prev_2014 = second_2014;
43
44         System.out.println("Penelusuran maju: ");
45         forwardTraversal_2014(head_2014);
46
47         System.out.println("Penelusuran mundur: ");
48         backwardTraversal_2014(third_2014);
49
50     }
51
52 }
53

```

2.4.3 Output

```

<terminated> PenelusuranDLL_2511532014 [Java Application] (
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

2.4.4 Analisis

Proses penelusuran membuktikan keunggulan DLL. Pada penelusuran mundur, kita tidak perlu melakukan perulangan dari awal list; cukup mulai dari simpul akhir (tail) dan mengikuti rantai prev hingga mencapai simpul yang memiliki penunjuk prev bernilai null.

BAB III

KESIMPULAN

Berdasarkan praktikum pekan 6, dapat disimpulkan bahwa:

1. Double Linked List memberikan fleksibilitas akses data dua arah melalui penunjuk next dan prev.
2. Setiap operasi modifikasi (tambah/hapus) memerlukan pembaharuan referensi pada kedua sisi simpul untuk menjaga keutuhan struktur data.
3. Penelusuran mundur (backward) dapat dilakukan dengan efisiensi yang sama dengan penelusuran maju selama referensi ke simpul akhir tersedia.

DAFTAR PUSTAKA

Laporan Tugas

1. Class Lagu_2511532014

a. Langkah Pengerjaan

- i. Mendefinisikan atribut judul_2014 dan penyanyi_2014 dengan akses private sebagai representasi data lagu.
- ii. Menambahkan dua penunjuk (pointer) yaitu next_2014 dan prev_2014 bertipe objek Lagu_2511532014.
- iii. Membuat konstruktor untuk menginisialisasi identitas lagu dan mengatur kedua penunjuk ke null.
- iv. Menyediakan metode Getter dan Setter untuk setiap atribut guna mendukung prinsip enkapsulasi.

b. Kode Program

```
1. package pekan6_2511532014;
2.
3. public class Lagu_2511532014 {
4.     private String judul_2014;
5.     private String penyanyi_2014;
6.
7.     Lagu_2511532014 next_2014;
8.     Lagu_2511532014 prev_2014;
9.
10.    // konstruktor
11.    public Lagu_2511532014(String judul_2014, String
    penyanyi_2014) {
12.        this.judul_2014 = judul_2014;
13.        this.penyanyi_2014 = penyanyi_2014;
14.        this.next_2014 = null;
15.        this.prev_2014 = null;
16.    }
17.
18.    // getter
19.    public String getJudul_2014() {
20.        return judul_2014;
21.    }
22.
23.    public String getPenyanyi_2014() {
24.        return penyanyi_2014;
25.    }
26.
27.    // setter
28.    public void setJudul_2014(String judul_2014) {
29.        this.judul_2014 = judul_2014;
30.    }
31.
32.    public void setPenyanyi_2014(String penyanyi_2014) {
33.        this.penyanyi_2014 = penyanyi_2014;
34.    }
35. }
36.
```


c. Analisis

Kelas Lagu_2511532014 dirancang sebagai struktur simpul (node) pada Double Linked List (DLL). Keberadaan dua penunjuk (next dan prev) memungkinkan data lagu dihubungkan secara dua arah. Analisis pada desain ini menunjukkan bahwa setiap objek lagu menyimpan referensi ke lagu sebelum dan sesudahnya dalam memori. Hal ini krusial untuk fitur playlist yang memungkinkan pengguna berpindah ke lagu selanjutnya (next) maupun kembali ke lagu sebelumnya (previous) tanpa harus mengulang pencarian dari awal antrian.

2. Class Musik_2511532014

a. Langkah Pengerjaan

- i. Mendeklarasikan variabel head_2014 dan tail_2014 untuk melacak ujung depan dan ujung belakang playlist.
- ii. Mengimplementasikan metode tambahLagu_2014 dengan logika Insert Tail, yaitu menyambungkan lagu baru ke tail dan memperbarui referensi prev.
- iii. Mengimplementasikan metode hapusLaguAwal_2014 dengan memindahkan head ke simpul selanjutnya dan memutuskan hubungan simpul lama.
- iv. Menyusun metode tampilMaju_2014 menggunakan penunjuk next dan tampilMundur_2014 menggunakan penunjuk prev.
- v. Mengimplementasikan metode cariLagu_2014 dengan penelusuran linear berdasarkan judul lagu.
- vi. Mengelola antarmuka interaktif melalui menu berbasis switch-case di dalam metode main.

b. Kode Program

```
1. package pekan6_2511532014;
2.
3. import java.util.Scanner;
4.
5. public class Musik_2511532014 {
6.
7.     Lagu_2511532014 head_2014;
8.     Lagu_2511532014 tail_2014;
9.
10.    // menu
11.    public static void tampilkanMenu_2014() {
12.        System.out.println("\n=== Playlist Musik NIM:
2511532014 ===");
13.        System.out.println("1. Tambah Lagu");
14.        System.out.println("2. Hapus Lagu Pertama");
15.        System.out.println("3. Lihat Playlist (Maju)");
16.        System.out.println("4. Lihat Playlist (Mundur)");
```

```

17.     System.out.println("5. Cari Lagu");
18.     System.out.println("6. Keluar");
19. }
20.
21.     // menambahkan lagu di akhir playlist
22.     public void tambahLagu_2014(String judul_2014, String
penyanyi_2014) {
23.         Lagu_2511532014 laguBaru_2014 = new
Lagu_2511532014(judul_2014, penyanyi_2014);
24.
25.         //jika playlist kosong
26.         if (head_2014 == null) {
27.             head_2014 = laguBaru_2014;
28.             tail_2014 = laguBaru_2014;
29.         } else {
30.             tail_2014.next_2014 = laguBaru_2014;
31.             laguBaru_2014.prev_2014 = tail_2014;
32.             tail_2014 = laguBaru_2014;
33.         }
34.         System.out.println("Lagu berhasil ditambahkan");
35.     }
36.
37.     // menghapus lagu pertama
38.     public void hapusLaguAwal_2014() {
39.         // Jika playlist kosong
40.         if (head_2014 == null) {
41.             System.out.println("Playlist kosong!");
42.             return;
43.         }
44.
45.         System.out.println("Lagu " + head_2014.getJudul_2014()
+ " berhasil dihapus.");
46.
47.         // Jika hanya ada 1 lagu
48.         if (head_2014 == tail_2014) {
49.             head_2014 = null;
50.             tail_2014 = null;
51.         } else {
52.             head_2014 = head_2014.next_2014;
53.             head_2014.prev_2014 = null;
54.         }
55.     }
56.
57.     // tampil playlist dari awal sampai akhir
58.     public void tampilMaju_2014() {
59.
60.         if (head_2014 == null) {
61.             System.out.println("Playlist kosong!");
62.             return;
63.         }
64.
65.         Lagu_2511532014 lagu_2014 = head_2014;
66.         System.out.println("\n=== Playlist Maju ===");
67.         while (lagu_2014 != null) {
68.             System.out.println("Judul      : " +
lagu_2014.getJudul_2014());

```

```

69.         System.out.println("Penyanyi : " +
lagu_2014.getPenyanyi_2014());
70.         System.out.println("-----");
71.
72.         lagu_2014 = lagu_2014.next_2014;
73.     }
74. }
75.
76. // tampil playlist dari akhir ke awal
77. public void tampilMundur_2014() {
78.     if (tail_2014 == null) {
79.         System.out.println("Playlist kosong!");
80.         return;
81.     }
82.     Lagu_2511532014 lagu_2014 = tail_2014;
83.     System.out.println("\n=== Playlist Mundur ===");
84.     while (lagu_2014 != null) {
85.         System.out.println("Judul      : " +
lagu_2014.getJudul_2014());
86.         System.out.println("Penyanyi : " +
lagu_2014.getPenyanyi_2014());
87.         System.out.println("-----");
88.         lagu_2014 = lagu_2014.prev_2014;
89.     }
90. }
91.
92. // mencari lagu berdasarkan judul
93. public void cariLagu_2014(String judulCari_2014) {
94.     if (head_2014 == null) {
95.         System.out.println("Playlist kosong!");
96.         return;
97.     }
98.     Lagu_2511532014 lagu_2014 = head_2014;
99.     boolean ditemukan_2014 = false;
100.    while (lagu_2014 != null) {
101.        if
(lagu_2014.getJudul_2014().equalsIgnoreCase(judulCari_2014)) {
102.            System.out.println("\nLagu ditemukan!");
103.            System.out.println("Judul      : " +
lagu_2014.getJudul_2014());
104.            System.out.println("Penyanyi : " +
lagu_2014.getPenyanyi_2014());
105.            ditemukan_2014 = true;
106.            break;
107.        }
108.        lagu_2014 = lagu_2014.next_2014;
109.    }
110.    if (!ditemukan_2014) {
111.        System.out.println("Lagu tidak ditemukan!");
112.    }
113. }
114.
115. // main program
116. public static void main(String[] args) {
117.     Scanner input_2014 = new
Scanner(System.in);

```

```

118.         Musik_2511532014 playlist_2014 = new
Musik_2511532014();
119.         int pilihan_2014;
120.         do {
121.             tampilkanMenu_2014();
122.             System.out.print("Pilihan: ");
123.             pilihan_2014 = input_2014.nextInt();
124.             input_2014.nextLine();
125.             switch (pilihan_2014) {
126.                 case 1:
127.                     System.out.print("Judul Lagu : ");
128.                     String judul_2014 =
input_2014.nextLine();
129.                     System.out.print("Penyanyi : ");
130.                     String penyanyi_2014 =
input_2014.nextLine();
131.                     playlist_2014.tambahLagu_2014(judul_2014, penyanyi_2014);
132.                     break;
133.                 case 2:
134.                     playlist_2014.hapusLaguAwal_2014();
135.                     break;
136.                 case 3:
137.                     playlist_2014.tampilMaju_2014();
138.                     break;
139.                 case 4:
140.                     playlist_2014.tampilMundur_2014();
141.                     break;
142.                 case 5:
143.                     System.out.print("Masukkan Judul
Lagu: ");
144.                     String cari_2014 =
input_2014.nextLine();
145.                     playlist_2014.cariLagu_2014(cari_2014);
146.                     break;
147.                 case 6:
148.                     System.out.println("Program
selesai.");
149.                     break;
150.                 default:
151.                     System.out.println("Pilihan tidak
valid!");
152.             }
153.         } while (pilihan_2014 != 6);
154.         input_2014.close();
155.     }
156.
157. }
158.

```

c. Output

```

<terminated> Musik_2511532014 [Java Application] C:\Users\asus\.p2\pool\plugins\
=== Playlist Musik NIM: 2511532014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu : Rule
Penyanyi : Ado
Lagu berhasil ditambahkan

=== Playlist Musik NIM: 2511532014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu : Ussewa
Penyanyi : Ado
Lagu berhasil ditambahkan

=== Playlist Musik NIM: 2511532014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist Maju ===

```

d. Analisis

Program ini mensimulasikan pemutar musik digital dengan menerapkan mekanisme Double Linked List. Analisis pada fitur tampilMundur_2014 menunjukkan efisiensi yang sangat baik; program langsung memulai iterasi dari variabel tail_2014 tanpa perlu mencari titik akhir terlebih dahulu. Hal ini dimungkinkan karena sifat DLL yang menyimpan alamat simpul sebelumnya.

Selain itu, pada metode hapusLaguAwal_2014, program melakukan pengecekan jika hanya tersisa satu lagu (dimana head == tail), maka kedua penunjuk diatur kembali ke null. Ketelitian dalam memperbarui dua penunjuk (next dan prev) secara bersamaan sangat penting; jika salah satu

penunjuk tidak diperbarui, akan terjadi broken link yang menyebabkan kegagalan navigasi atau `NullPointerException`. Implementasi ini jauh lebih fleksibel dibanding Single Linked List karena mendukung navigasi penuh di dalam daftar putar.